

Dockerfile Avanzado

Arturo Silvelo

Try New Roads

Volumes

Los **volúmenes** son almacenes de datos persistentes para contenedores, creados y gestionados por Docker. Puedes crear un volumen explícitamente con el comando: `docker volume create`, o Docker puede crearlo durante la creación de un contenedor o servicio.

Gestión de volumes

Cuando se crea un **volumen** se almacena en un directorio del **host Docker**. Cuando se monta un volumen en un contenedor, ese directorio es lo que se monta dentro del contenedor.

Esto es similar a como funcionan los **bind mounts**, excepto que los volúmenes son gestionados por Docker y están aislados de la funcionalidad principal del sistema host.

Ubicación de almacenamiento:

- **Linux:** `/var/lib/docker/volumes/<volume-name>/_data`
- **Windows (Docker Desktop):** `\\wsl$\\docker-desktop-data\\data\\docker\\volumes\\<volume-name>\\_data`

- Opción 1

```
services:
  app-base:
    build:
      context: ../../../../app
      dockerfile: ../02-compose/ejemplos/Dockerfile
    ports:
      - "3001:3000"
    image: app-base
    container_name: app-base
    environment:
      NODE_ENV: development
      PORT: 3000
    volumes:
      - app-uploads-compose:/app/uploads

volumes:
  app-uploads-compose:
    driver: local
    name: app-uploads-compose
```

```
docker compose -f 03-volumes/ejemplos/1.volume/compose.yml up -d --build
```

- Opción 2

```
docker volume create app-uploads
docker run -p3000:3000 --name app-base -v app-uploads:/app/uploads app-base
```

- Verificación

```
docker volume ls
ls /var/lib/docker/volumes/
```

```
curl -X POST http://localhost:3001/upload \
-F "file=@./03-volumes/ejemplos/lenna.png" \
-H "Content-Type: multipart/form-data"
```

- Resultado

```
docker exec app-base-compose ls /app/uploads
docker exec app-base ls /app/uploads
```

```
ls /var/lib/docker/volumes/app-uploads/_data
ls /var/lib/docker/volumes/app-uploads-compose/_data
```

- Limpiar

```
docker rm -f app-base app-base-compose
docker volume rm app-uploads-compose app-uploads
```

Ciclo de vida de volumes

- Un volume dado puede ser montado en **múltiples contenedores simultáneamente**
- Cuando ningún contenedor está usando un volume, el volume sigue **disponible para Docker**
- Puedes eliminar volumes no utilizados usando: `docker volume prune`

Cuando usar volumes

- Se pueden gestionar usando comandos CLI de Docker o la API
- Funcionan tanto en contenedores Linux como Windows
- Se pueden compartir más seguramente entre múltiples contenedores
- Cuando tu aplicación requiere I/O de alto rendimiento
- Fáciles de migrar y hacer backup
- Se necesita tener contenido previo en el contenedor.

Named y Anonymous volumes

Un volumen puede ser **nombrado** o **anónimo**. Los volúmenes anónimos reciben un nombre aleatorio que está garantizado de ser único dentro de un host Docker dado.

Al igual que los volúmenes nombrados, **los volúmenes anónimos persisten** incluso si eliminas el contenedor que los usa, **excepto** si usas el flag `--rm` al crear el contenedor, en cuyo caso el volumen anónimo asociado con el contenedor es destruido.

Características de Anonymous volumes

Si creas múltiples contenedores consecutivamente que cada uno usa volúmenes anónimos, **cada contenedor crea su propio volumen.**

Los volúmenes anónimos **no son reutilizados o compartidos** entre contenedores automáticamente.

- compose.yml

```
x-app-base: &app-base
  build:
    context: ../../../../app
    dockerfile: ../03-volumes/ejemplos/Dockerfile.anonymous
  ports:
    - "3000:3000"
  image: app-base
  container_name: app-base
  environment:
    NODE_ENV: development
    PORT: 3000

services:
  app-base-1:
    <<: *app-base
  app-base-2:
    <<: *app-base
    container_name: app-base-2
    ports:
      - "3002:3000"
  app-base-3:
    <<: *app-base
    container_name: app-base-3
    ports:
      - "3003:3000"
```

- Dockerfile

```
VOLUME /app/uploads
VOLUME /app/logs
```

- Ejecución

```
docker compose -f 03-volumes/ejemplos/2.anonymous/compose.yml up -d --build
```

- Verificación

```
docker volume ls
DRIVER      VOLUME NAME
local       4c6d0a83e7cc576945aa7eca1a931af58a6568b7a9e4145d3148742d57d16390
local       4c9f5b83ced73c1a8974be98b9b217be1aa4ae31cc2601c8e5834ee898fe9b0d
local       603d9377cd6a6e74cdb0d77b927c03b9775f3e01e0beff86449bd477f1de8e51
local       a4a41d61068c9e18ac0ef0e4a20fbefb32ef158085b5f55ac933cab69a76e151
local       b24b0bd309a53b5d6c9dd03270390e51b650524818146cd121dbc921f2732578
local       e5c6355e0cfd95a064cf05be083960b067e95a7b894390bfff1ccaa064baba6ce
```

- Limpiar

```
docker compose -f 03-volumes/ejemplos/2.anonymous/compose.yml down -v
# ó
docker compose -f 03-volumes/ejemplos/2.anonymous/compose.yml down
docker volume prune
```

```
docker volume ls
DRIVER      VOLUME NAME
```

Montaje de volumes: --mount vs --volume

Para montar un volumen con el comando `docker run`, puedes usar los flags `--mount` o `--volume`.

En general, `--mount` es preferido. La principal diferencia es que el flag `--mount` es más explícita y soporta todas las opciones disponibles.

Debes usar `--mount` si quieres:

- Especificar opciones del driver de volumen
- Montar un subdirectorio de un volumen
- Montar un volumen en un servicio de Swarm

```
docker run -d --mount source=app-uploads-mount,target=/app/logs,ro --name app-base-mount app-base
docker run -d -v app-uploads-volume:/app/logs:ro --name app-base-volume app-base
```

```
docker volume ls
DRIVER      VOLUME NAME
local      86d082cdbabb590be11db783dde3be9f039d38ba10c1df587a725323a68eeca5
local      983d94bf98ca23ec46acfccbc00543edc9e6ec5df71eac6d86021daa9f1f1149
local      app-uploads-mount
local      app-uploads-volume
```

```
docker rm -f --volumes app-base-mount app-base-volume
docker volume rm app-uploads-volume app-uploads-mount
```

Montar un subdirectorio

- Creación y configuración del volumen

```
docker volume create logs
docker run --rm \
  --mount src=logs,dst=/logs \
  alpine mkdir -p /logs/app1 /logs/app2
```

- Crear contenedores

```
docker run -d --mount source=logs,target=/app/logs,volume-subpath=app1 --name app-base-1 app-base
docker run -d --mount source=logs,target=/app/logs,volume-subpath=app2 --name app-base-2 app-base
```

- Verificar

```
docker volume ls
DRIVER      VOLUME NAME
local      2ddcf36b0b174340a774fdc991d3c52c4d20d8d37ed8ded7f0038214990501ed
local      c4e4afc0cc8c8be3832893e24d5872483178fd8003a1942638517537b15a796c
local      logs
```

- Verificar

```
docker run --rm --mount src=logs,dst=/logs alpine cat /logs/app1/app.log
docker run --rm --mount src=logs,dst=/logs alpine cat /logs/app2/app.log
```

- Limpiar

```
docker rm -f --volumes app-base-1 app-base-2
docker volume rm logs
```

```
x-app-base: &app-base
  build:
    context: ../../../../app
    dockerfile: ../03-volumes/ejemplos/Dockerfile.anonymous
  ports:
    - "3000:3000"
  image: app-base
  container_name: app-base
  environment:
    NODE_ENV: development
    PORT: 3000
  depends_on:
    - init-logs
x-alpine: &alpine
  image: alpine
  volumes:
    - app-logs:/logs
  command: |
    sh -c "
      if [ ! -d '/logs/app1' ]; then
        mkdir -p /logs/app1 /logs/app2
        echo 'Directorios creados'
      else
        echo 'Directorios ya existen'
      fi
    "
  restart: "no"

services:
  app-base-1:
    <<: *app-base
    container_name: app-base-1
    volumes:
      - type: volume
        source: app-logs
        target: /app/logs
        volume:
          subpath: app1
  app-base-2:
    <<: *app-base
    container_name: app-base-2
    ports:
      - "3001:3000"
    volumes:
      - type: volume
        source: app-logs
        target: /app/logs
        volume:
          subpath: app2
  init-logs:
    <<: *alpine
  test:
    <<: *alpine
    command: |
      sh -c "
        ls -l /logs/app1
        cat /logs/app1/app.log
        ls -l /logs/app2
        cat /logs/app2/app.log
      "
    depends_on:
      - app-base-1
      - app-base-2
  volumes:
    app-logs:
      driver: local
      name: app-logs
```

- Ejecución

```
docker compose -f 03-volumes/ejemplos/3.mount/compose.yaml up -d --build
```

- Validación

```
docker compose -f 03-volumes/ejemplos/3.mount/compose.yaml logs test
```

- Limpiar

```
docker compose -f 03-volumes/ejemplos/3.mount/compose.yaml down -v
docker volume rm app-logs
```

Compartir datos entre máquinas

Al construir aplicaciones tolerantes a fallos, puedes necesitar configurar múltiples réplicas del mismo servicio para que tengan acceso a los mismos archivos.

- Añadir lógica a tu aplicación para almacenar archivos en un sistema de almacenamiento de objetos en la nube como **Amazon S3**.
- Crear volúmenes con un **driver** que soporte escribir archivos a un sistema de almacenamiento externo como **NFS** o **Amazon S3**.

